

Testing and Continuous Integration Lab Report

Phase 1 Test Coverage
Phase 2 Test Driven Development
Phase 3 Continuous Integration

Student	Sokrat Rostomyan
GitHub	rosto-designs
Team repository	kenner1-unlv/Group-2
Public fork	rosto-designs/Group-2
Course	CS 472 Software Testing and Continuous Integration
Report date	September 18, 2026

This report documents my individual contributions across all three lab phases. It provides direct links to issues, pull requests, commits, CI runs, merge commits, and peer reviews, together with local test and coverage excerpts. All three of my contributions were independently reviewed, approved, and merged into the team repository.

Evidence Summary

Phase	Contribution	Result	PR
1	Valid account withdrawal test	5 passed 84% total coverage	PR 72
2	GET counters by name using TDD	RED reproduced GREEN 3 passed 96%	PR 75
3	80% coverage quality gate	Controlled failure then 22 passed at 94.85%	PR 85

Phase 1 issue: [Open evidence](#)

Phase 2 issue: [Open evidence](#)

Phase 3 issue: [Open evidence](#)

Phase 1 reviewer approval: [Open evidence](#)

Phase 2 reviewer approval: [Open evidence](#)

Phase 3 reviewer approval: [Open evidence](#)

Phase 1 Test Coverage

I added a focused test for a valid account withdrawal. The test starts with a balance of 100.0, withdraws 30.0, and verifies that the remaining balance is 70.0 using `pytest.approx()`. This directly exercises the successful path of `Account.withdraw()`.

Setup and Assignment

Team Test Coverage setup issue: [Open evidence](#)

Team Test Coverage setup PR: [Open evidence](#)

Setup commit: [Open evidence](#)

My Phase 1 issue and acceptance criteria: [Open evidence](#)

My Phase 1 PR: [Open evidence](#)

Test code and PR diff: [Open evidence](#)

Local Setup Baseline

The baseline run on my machine used the `test_coverage_lab` directory and completed successfully with four tests. It reported 81% total coverage and 62% coverage for `models/account.py`.

```
===== test session starts =====
platform linux -- Python 3.12.3, pytest-8.4.2, pluggy-1.6.0 -- /home/soko/Group-2/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/soko/Group-2/test_coverage_lab
configfile: pytest.ini
plugins: cov-7.1.0
collected 4 items

tests/test_account.py::test_account_role_assignment PASSED [ 25%]
tests/test_account.py::test_invalid_role_assignment PASSED [ 50%]
tests/test_account.py::test_password_hashing PASSED [ 75%]
tests/test_account.py::test_account_deactivation_and_reactivation PASSED [100%]

===== tests coverage =====
----- coverage: platform linux, python 3.12.3-final-0 -----
Name                               Stmts  Miss  Cover   Missing
-----
models/__init__.py                   7      0  100%
models/account.py                   58     22   62%    30, 34, 47-49, 53-56, 60-62, 66-68, 72-76, 102-103
tests/__init__.py                    0      0  100%
tests/test_account.py               49      0  100%
-----
TOTAL                               114     22   81%
===== 4 passed in 1.00s =====
```

Figure 1. Phase 1 baseline on my machine: `test_coverage_lab`, four tests passed, 81% total coverage.

Coverage Result After My Test

After adding `test_valid_withdrawal`, all five tests passed. Total coverage increased from 81% to 84%, while coverage of `models/account.py` increased from 62% to 67%.

```

===== test session starts =====
platform linux -- Python 3.12.3, pytest-8.4.2, pluggy-1.6.0 -- /home/soko/Group-2/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/soko/Group-2/test_coverage_lab
configfile: pytest.ini
plugins: cov-7.1.0
collected 5 items

tests/test_account.py::test_account_role_assignment PASSED [ 20%]
tests/test_account.py::test_invalid_role_assignment PASSED [ 40%]
tests/test_account.py::test_password_hashing PASSED [ 60%]
tests/test_account.py::test_account_deactivation_and_reactivation PASSED [ 80%]
tests/test_account.py::test_valid_withdrawal PASSED [100%]

===== tests coverage =====
----- coverage: platform linux, python 3.12.3-final-0 -----

```

Name	Stmts	Miss	Cover	Missing
models/__init__.py	7	0	100%	
models/account.py	58	19	67%	30, 34, 47-49, 53-56, 60-62, 66-68, 73, 75, 102-103
tests/__init__.py	0	0	100%	
tests/test_account.py	55	0	100%	
TOTAL	120	19	84%	

```

===== 5 passed in 1.00s =====

```

Figure 2. Phase 1 final local result: five tests passed, including test_valid_withdrawal, with 84% total coverage.

Reviewer Feedback and Merge

Russell Kennedy independently ran the targeted test and complete Phase 1 suite. He confirmed the 67% model coverage, 84% total coverage, focused scope, specific balance assertion, and successful CI result. The review approved the contribution without requesting corrective changes, so I retained the verified implementation and preserved the evidence in the PR.

- Reviewer approval: [Open evidence](#)
- Successful starter CI run 37: [Open evidence](#)
- Approved and merged PR: [Open evidence](#)
- Merge commit: [Open evidence](#)

Peer Review Performed

I reviewed Glen Testamark's Phase 1 PR, which covered account serialization and required-field validation. I examined the test diff and local pytest/coverage results. My initial review identified that the serialization test checked keys but did not verify returned field values. Glen subsequently added checks for the serialized name and email and removed an irrelevant assertion. I reran the proposed merge, confirmed seven passing tests and 90% total coverage, and approved the updated work.

- Teammate PR reviewed: [Open evidence](#)
- My final Phase 1 review: [Open evidence](#)

Phase 2 Test Driven Development

I implemented retrieval of an existing counter through GET /counters/<name>. The test creates a named counter, retrieves it, checks for HTTP 200, and verifies the returned JSON mapping. The implementation was developed in ordered RED and GREEN commits.

Setup and Assignment

Team TDD setup issue: [Open evidence](#)

Team TDD setup PR: [Open evidence](#)

Setup commit: [Open evidence](#)

My Phase 2 issue and acceptance criteria: [Open evidence](#)

My Phase 2 PR: [Open evidence](#)

Test and endpoint diff: [Open evidence](#)

Local Setup Baseline

The starter tdd_lab suite ran successfully on my machine with two passing tests and 96% total coverage before I introduced the retrieval test.

```
===== test session starts =====
platform linux -- Python 3.12.3, pytest-8.4.2, pluggy-1.6.0 -- /home/soko/Group-2/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/soko/Group-2/tdd_lab
configfile: pytest.ini
plugins: cov-7.1.0
collected 2 items

tests/test_counter.py::TestCounterEndpoints::test_create_counter PASSED [ 50%]
tests/test_counter.py::TestCounterEndpoints::test_list_counters PASSED [100%]

===== tests coverage =====
----- coverage: platform linux, python 3.12.3-final-0 -----
Name                Stmts   Miss  Cover   Missing
-----
src/__init__.py        3      0   100%
src/counter.py       15      1    93%    19
src/status.py         6      0   100%
-----
TOTAL                  24      1    96%
===== 2 passed in 0.33s =====
```

Figure 3. Phase 2 setup evidence: tdd_lab on my machine with two baseline tests passing.

RED Phase

Commit fa5827b added the retrieval test before the endpoint existed. The targeted test failed as intended because Flask returned HTTP 405 Method Not Allowed rather than HTTP 200. This demonstrated that the test detected the missing behavior.

RED commit: [Open evidence](#)

```

===== test session starts =====
platform linux -- Python 3.12.3, pytest-8.4.2, pluggy-1.6.0 -- /home/soko/Group-2/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/soko/Group-2/tdd_lab
configfile: pytest.ini
plugins: cov-7.1.0
collected 1 item

tests/test_counter.py::TestCounterEndpoints::test_retrieve_existing_counter FAILED [100%]

===== FAILURES =====
----- TestCounterEndpoints.test_retrieve_existing_counter -----

self = <tests.test_counter.TestCounterEndpoints object at 0x774ba008ef00>
client = <FlaskClient <Flask 'src.counter'>>

    def test_retrieve_existing_counter(self, client):
        """It should retrieve an existing counter by name"""
        counter_name = "retrievable"
        create_result = client.post(f'/counters/{counter_name}')
        assert create_result.status_code == status.HTTP_201_CREATED

        result = client.get(f'/counters/{counter_name}')

>       assert result.status_code == status.HTTP_200_OK
E       assert 405 == 200
E         + where 405 = <WrapperTestResponse streamed [405 METHOD NOT ALLOWED]>.status_code
E         + and    200 = status.HTTP_200_OK

tests/test_counter.py:54: AssertionError
===== tests coverage =====
----- coverage: platform linux, python 3.12.3-final-0 -----

Name                Stmts   Miss  Cover   Missing
-----
src/__init__.py        3      0   100%
src/counter.py       15      2    87%   19, 26
src/status.py         6      0   100%
TOTAL                 24      2    92%

===== short test summary info =====
FAILED tests/test_counter.py::TestCounterEndpoints::test_retrieve_existing_counter - assert 405 == 200
===== 1 failed in 0.32s =====

```

Figure 4. RED evidence: the new retrieval test failed with 405 instead of the expected 200.

GREEN and REFACTOR Phases

Commit 60dc341 added the minimum route needed to retrieve the named counter. The targeted test and full suite then passed, with three tests and 96% total coverage. I evaluated the implementation for refactoring, but the handler was already a single focused return using the existing counter store. An additional refactor would have added complexity without improving clarity, so the code was retained as the refactored state.

GREEN commit: [Open evidence](#)

```

===== test session starts =====
platform linux -- Python 3.12.3, pytest-8.4.2, pluggy-1.6.0 -- /home/soko/Group-2/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/soko/Group-2/tdd_lab
configfile: pytest.ini
plugins: cov-7.1.0
collected 3 items

tests/test_counter.py::TestCounterEndpoints::test_create_counter PASSED [ 33%]
tests/test_counter.py::TestCounterEndpoints::test_list_counters PASSED [ 66%]
tests/test_counter.py::TestCounterEndpoints::test_retrieve_existing_counter PASSED [100%]

===== tests coverage =====
----- coverage: platform linux, python 3.12.3-final-0 -----
Name                Stmts   Miss  Cover   Missing
-----
src/__init__.py        3      0   100%
src/counter.py        18      1    94%    19
src/status.py          6      0   100%
-----
TOTAL                  27      1    96%
===== 3 passed in 0.26s =====

```

Figure 5. GREEN and final evidence: three tests passed with 96% total coverage.

Reviewer Feedback and Merge

Russell independently reproduced the RED failure and GREEN success. He confirmed that the test verified both the status code and returned JSON, that the implementation was minimal, and that the full suite passed at 96% coverage. He accepted my explanation that no additional refactor was necessary and approved the PR without requesting changes.

Reviewer approval: [Open evidence](#)

Approved and merged PR: [Open evidence](#)

Merge commit: [Open evidence](#)

Peer Review Performed

I reviewed Glen's DELETE-counter feature. I checked the source and test diffs, reproduced the RED failure at commit 325a7bf (405 instead of 204), reproduced the GREEN success at commit 160f6b5, and ran the complete proposed merge. The final suite passed four tests with 97% total coverage. I also noted that a 204 response discards the JSON body, but treated that as non-blocking because the assigned delete behavior and test passed.

Teammate PR reviewed: [Open evidence](#)

My final Phase 2 review: [Open evidence](#)

Phase 3 Continuous Integration

I updated the root GitHub Actions workflow so CI fails whenever total test coverage is below 80%. I demonstrated the gate through a controlled 100% threshold failure, diagnosed that failure from the Actions log, restored the required 80% threshold, and verified a successful rerun with all 22 tests passing at 94.85% coverage.

Repository and Synchronization Evidence

Public fork: [Open evidence](#)

Starter workflow success for my Phase 1 branch: [Open evidence](#) - successful run 37 before Phase 3

Before creating the report branch, I fetched upstream, fast-forwarded local main, pushed the synchronized state to my fork, and then fast-forwarded the report branch. The following excerpt shows the four relevant references at the same commit.

```
git log -1 --oneline main
git log -1 --oneline upstream/main
git log -1 --oneline origin/main
git log -1 --oneline HEAD

7cb2489  main
7cb2489  upstream/main
7cb2489  origin/main
7cb2489  add-testing-ci-report-rosto-designs
```

Issue PR and Final Workflow

Phase 3 issue and acceptance criteria: [Open evidence](#)

Combined CI PR: [Open evidence](#)

Final workflow diff: [Open evidence](#)

The final workflow executes the Phase 3 tests and enforces the required threshold:

```
- name: Run Tests with Pytest
  run: pytest --cov=src --cov-report=term-missing --cov-fail-under=80
```

Controlled Failure and Root Cause

For the controlled failure, commit aff33c3 temporarily set `--cov-fail-under=100`. All 22 tests passed, but measured coverage was 94.85%, so `pytest-cov` returned exit code 1 and GitHub Actions marked the job as failed. The failure was therefore caused by the deliberately unattainable coverage threshold, not by a failing test.

Controlled-failure commit: [Open evidence](#)

Failed Actions run: [Open evidence](#)

Failed job log: [Open evidence](#)

← CI Workflow

❌ ci: demonstrate coverage gate failure #1

Re-run jobs Latest #2

Summary

All jobs

build

Run details

Usage

Workflow file

Workflow file for this run

[.github/workflows/ci.yml](#) at [aff33c3](#)

```
1 name: CI Workflow # Name of the workflow
2
3 on:
4   push:
5   pull_request:
6
7 jobs:
8   build:
9     runs-on: ubuntu-latest # Use the latest Ubuntu runner
10    defaults:
11      run:
12        working-directory: ci_lab # Set the default working directory for all steps
13
14    steps:
15      - name: Checkout Repository
16        uses: actions/checkout@v4
17      - name: Set Up Python
18        uses: actions/setup-python@v5
19        with:
20          python-version: "3.9" # Ensure consistency in Python version
21
22      - name: Install Dependencies
23        run: |
24          python -m pip install --upgrade pip
25          pip install -r requirements.txt
26
27      - name: Run Tests with Pytest
28        run: pytest --cov=src --cov-report=term-missing --cov-fail-under=100
29
30      - name: Install Flake8
31        run: pip install flake8
32
33      - name: Run Flake8 Linting
34        run: flake8 src --count --select=E9,F63,F7,F82 --show-source --statistics
```

Figure 6. Controlled workflow version using a temporary 100% coverage threshold.

← CI Workflow

❌ ci: demonstrate coverage gate failure #1 Re-run jobs Latest #3 ...

Summary

All jobs

❌ build

Run details

Usage

Workflow file

Re-run triggered 2 minutes ago

rosto-designs [aff33c3](#) [phase3-coverage-gate-rosto-...](#) **Failure**

Total duration **20s** Artifacts **—**

ci.yml
on: push

❌ build 17s

Annotations
1 error and 1 warning

❌ build
Process completed with exit code 1.

⚠️ build
Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to ...
[Show more](#)

Figure 7. GitHub Actions correctly marked the controlled coverage-gate run as failed.

```

44  ===== tests coverage =====
45  _____ coverage: platform linux, python 3.9.25-final-0 _____
46
47  Name           Stmts   Miss  Cover   Missing
48  -----
49  src/__init__.py    3      0   100%
50  src/counter.py    87      5    94%   71, 85, 96, 99-100
51  src/status.py      7      0   100%
52  -----
53  TOTAL             97      5    95%
54  FAIL Required test coverage of 100% not reached. Total coverage: 94.85%
55  ===== 22 passed in 0.32s =====
56  Error: Process completed with exit code 1.

```

Figure 8. Failure diagnosis: 94.85% coverage did not meet the temporary 100% threshold.

Correction and Successful Rerun

Commit 386a6db restored the required threshold to 80%. The subsequent run succeeded because the measured 94.85% coverage exceeded the quality gate. The fork run and upstream PR-triggered run both completed successfully.

Fix commit: [Open evidence](#)

Successful fork run: [Open evidence](#)

Successful upstream PR run 48: [Open evidence](#)

The screenshot shows the GitHub Actions interface for a workflow named "ci: enforce 80 percent coverage threshold #2". The workflow is in a "Success" state, indicated by a green checkmark. The summary section shows that the workflow was re-run triggered now, with a total duration of 26s. The workflow file is "ci.yml" and it is triggered on "push". The workflow consists of a single job named "build" which completed successfully in 21s. There is a warning annotation stating "Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to ...".

← CI Workflow

✓ **ci: enforce 80 percent coverage threshold #2** Re-run all jobs Latest #2 ...

Summary

All jobs ⌵

✓ **build**

Run details

🕒 Usage

📄 Workflow file

Re-run triggered now

rosto-designs → 386a6db phase3-coverage-gate-rosto-... **Success**

Total duration **26s** Artifacts **—**

ci.yml
on: push

✓ **build** 21s

Annotations
1 warning

⚠️ **build**
Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to ...
[Show more](#)

Figure 9. Successful workflow summary after restoring the 80% threshold.

```

42  ===== tests coverage =====
43  _____ coverage: platform linux, python 3.9.25-final-0 _____
44
45  Name                Stmts   Miss  Cover   Missing
46  -----
47  src/__init__.py        3      0   100%
48  src/counter.py        87      5    94%   71, 85, 96, 99-100
49  src/status.py          7      0   100%
50  -----
51  TOTAL                  97      5    95%
52  Required test coverage of 80% reached. Total coverage: 94.85%
53  ===== 22 passed in 0.28s =====

```

Figure 10. Successful CI log: 22 tests passed and 94.85% coverage exceeded the 80% gate.

Reviewer Feedback Merge and Peer Review

Russell verified the final workflow against issue 70, confirmed that the controlled failure was independent of test success, and confirmed that restoring the 80% threshold returned CI to green. He approved the focused one-line workflow change without requesting corrections.

Reviewer approval: [Open evidence](#)

Approved and merged PR: [Open evidence](#)

Merge commit: [Open evidence](#)

For my assigned Phase 3 peer review, I examined Glen's workflow diff, controlled artifact-upload failure, correction that generated coverage.json, and the proposed merge. I independently ran the final coverage command: all 22 tests passed, the 80% threshold was reached at 94.85%, and the generated JSON contained valid totals. I approved the PR after confirming the artifact path and absence of regressions.

Teammate PR reviewed: [Open evidence](#)

My Phase 3 review: [Open evidence](#)

Reflection

Continuous Integration and Automated Testing

This lab made continuous integration concrete for me. A local passing test run is useful, but CI provides a repeatable check in a clean environment for every push and pull request. The workflow records the exact command, environment, result, and commit, which makes the evidence easier for another developer to reproduce. I also learned that automation is most useful when it enforces an explicit team expectation instead of merely displaying information.

Coverage and Quality Gates

Coverage helped identify which production statements were exercised, but the Phase 1 work also showed that a high percentage does not prove that assertions are meaningful. A test must check the correct behavior, such as the exact balance after a withdrawal or the exact JSON returned by an endpoint. The Phase 3 threshold converted coverage from a passive metric into a quality gate: a change below 80% now fails automatically. The controlled 100% experiment made that distinction visible because every test passed while the workflow still failed.

Failure Diagnosis

The Actions log provided the fastest route to the root cause. The line stating that 94.85% was below the required 100% separated a coverage-policy failure from a test failure. Reading the first decisive error rather than reacting only to the red status made the correction small and targeted: restore the intended 80% threshold and rerun the same workflow.

Technical Peer Review and Collaboration

Reviewing another student's work required more than checking whether GitHub displayed a green check. I inspected diffs, checked issue scope, reproduced tests locally, examined ordered TDD commits, and compared workflow behavior with the acceptance criteria. In return, Russell's reviews independently confirmed my results and gave the team confidence that the evidence was not dependent on my machine. The merge-conflict work also reinforced the importance of syncing with upstream and testing the proposed merge rather than only the contributor's branch.

Overall Lesson

The three phases form one development loop: write a behavior-focused test, use RED and GREEN results to guide implementation, enforce the shared standard in CI, diagnose failures from logs, and use peer review to verify both code and evidence. The result is not only working code but a traceable process that another team member can inspect and reproduce.

End of report. All hyperlinks were current when this report was prepared on September 18, 2026.